

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學

Tutorial 2

Build a Literature Search Tool

COMP5511 Artificial Intelligence Concepts

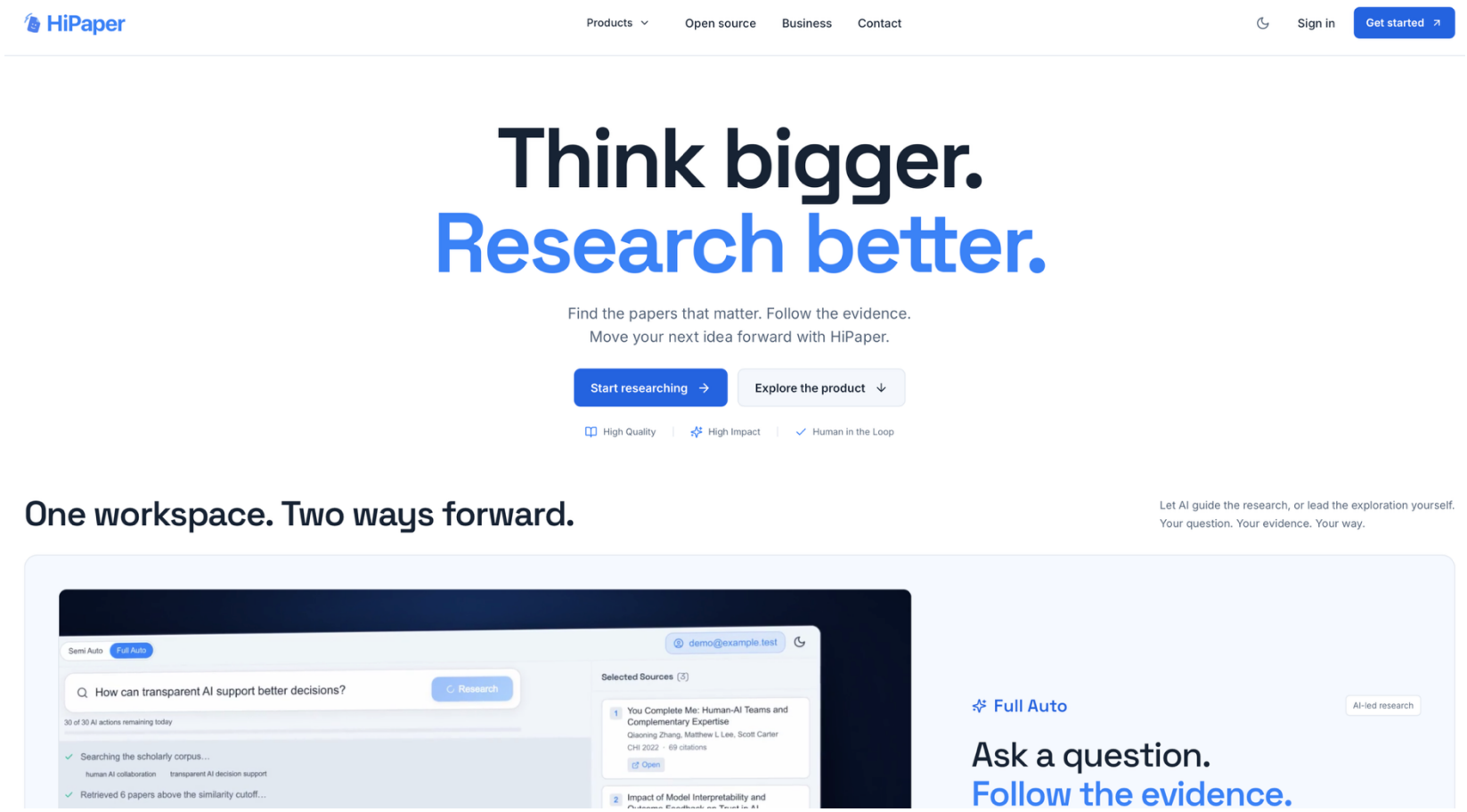
Pangjing Wu

Department of Computing

pang-jing.wu@connect.polyu.hk

HiPaper live demo

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學

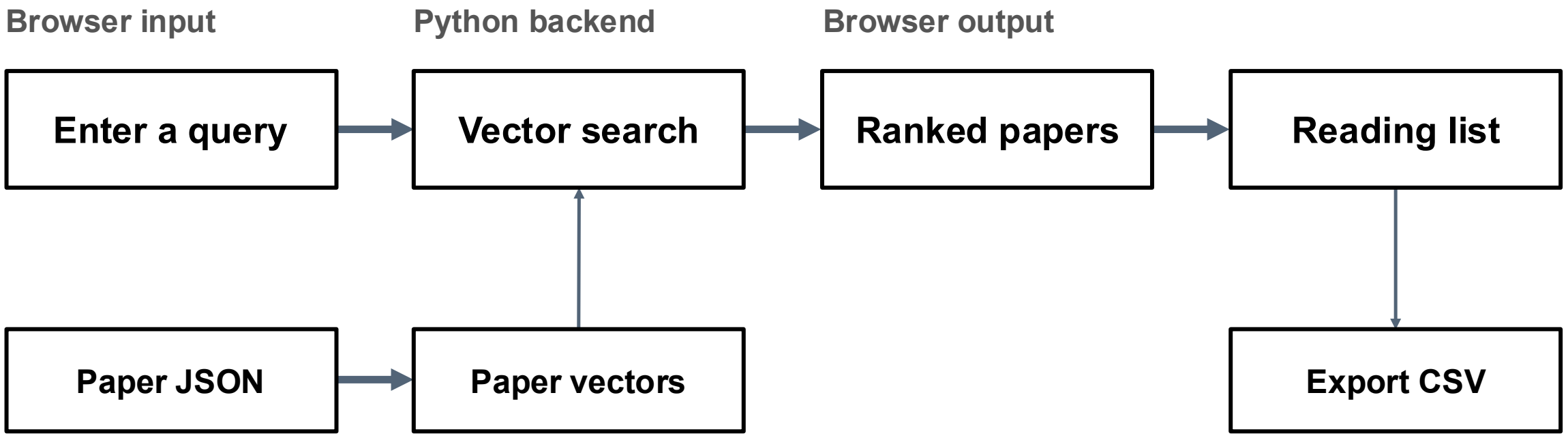


www.academic.hk

A simplified HiPaper

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學

Find papers by topic, save useful results and export a reading list as CSV.

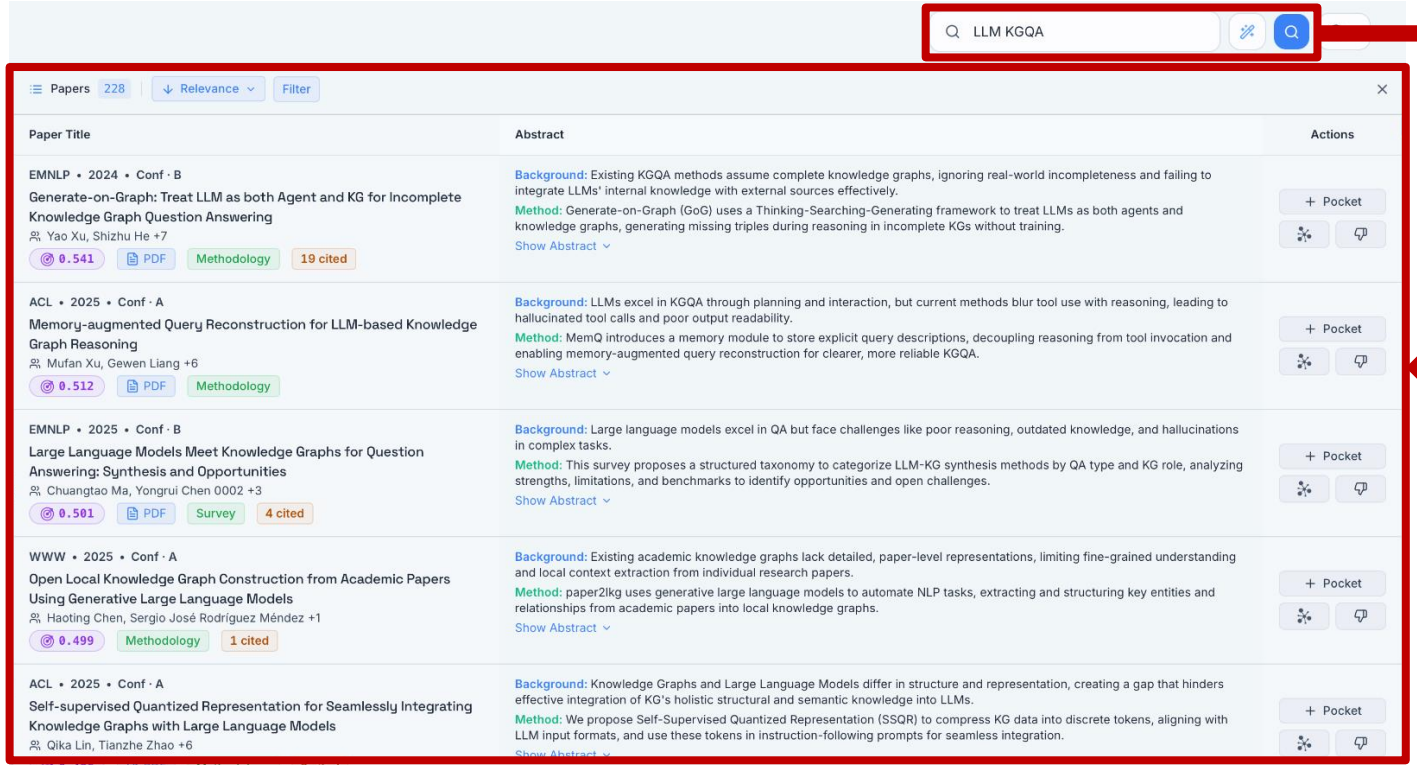


Frontend and backend

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學

Frontend

Browser interface



1 Query

2

3 Results

Backend

Server-side logic and data

Search API

Retrieval algorithm

Paper data

Cosine similarity

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學

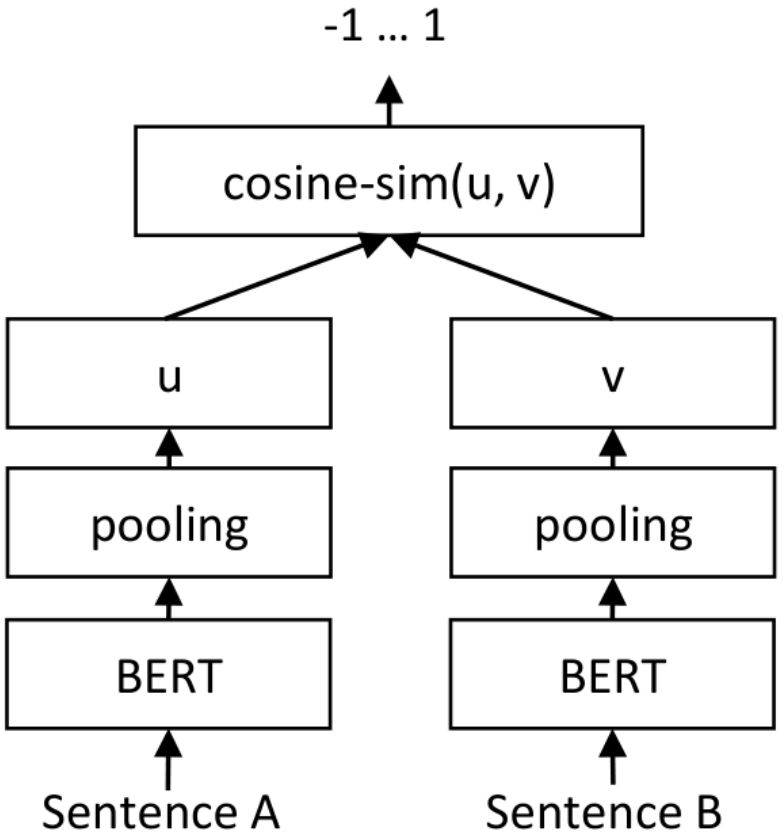
$$\cos(q, d) = (q \cdot d) / (\|q\|_2 \times \|d\|_2)$$

Query "How does Python list indexing work?" q = (1, 0)

Document text	Toy vector	Cosine with q
A Python, List Indexing List indexing uses zero-based positions.	(3, 4)	(1 × 3 + 0 × 4) / (1 × 5) = 0.60
B Classroom Feedback Feedback is collected through surveys.	(0, 1)	(1 × 0 + 0 × 1) / (1 × 1) = 0.00

These toy vectors make the calculation easy to do by hand. The demo uses 120 paper records.

A dual-encoder embedding model



Two inputs

Use Sentence A for the query and Sentence B for the paper text.

Two embeddings

The encoder and pooling produce one vector for each input.

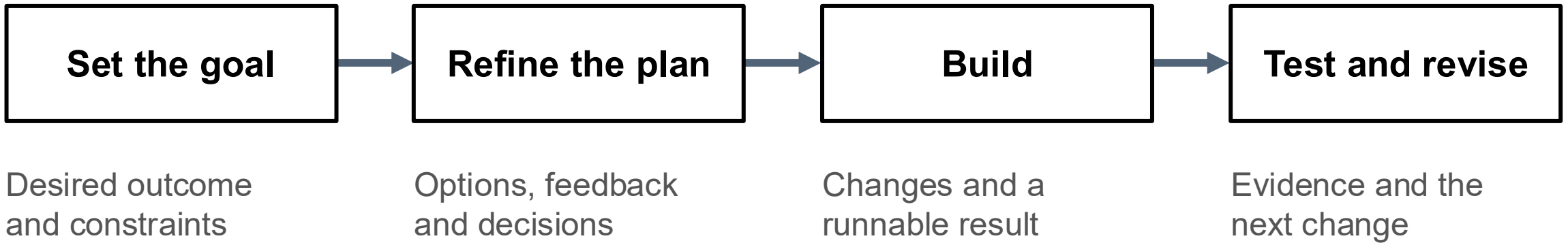
One similarity score

Compare the two vectors with cosine similarity.

Original Figure 2 Reimers and Gurevych, Sentence-BERT (2019)

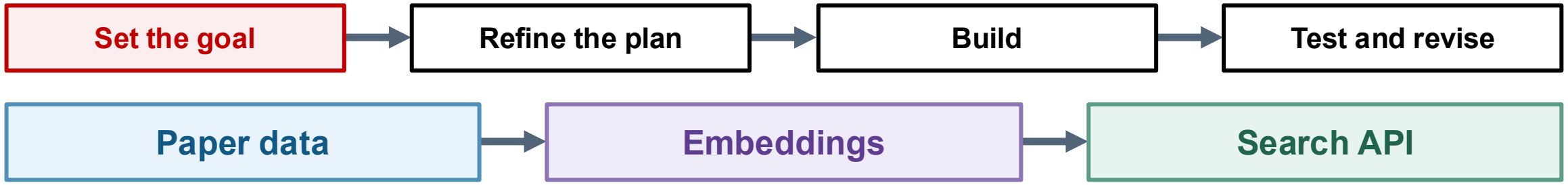
A Vibe Coding workflow

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學



1. Draft the architecture

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學



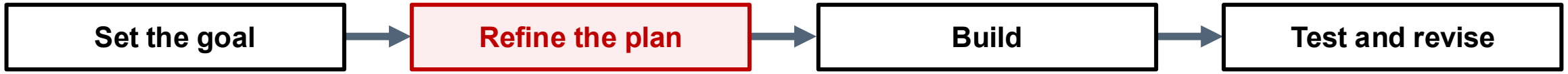
I'm building a local paper search tool with a reading list and CSV export. I have 120 records in **data/papers.json** and will use Python 3.11 in .venv.

Sketch the backend. Show how **paper data** flows through **embeddings** to the **search API**.

Use one diagram and up to three bullets. Wait for my go-ahead before editing.

Why I confirm the retrieval design

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學



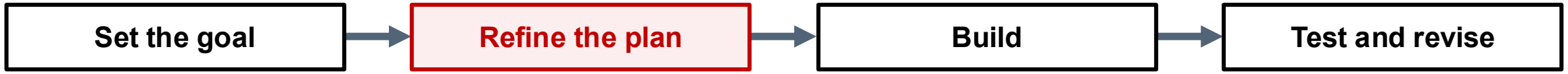
I've asked for a backend sketch. The retrieval settings are still open.

Open decision	What the agent might choose		Effect in the demo
Embedding model	Pick a model that requires a GPU	→	The demo can't start on CPU.
Paper vectors	Re-encode all 120 papers per search	→	Every query waits for 120 extra embeddings.
Result limit	Return all 120 papers every time	→	I still have 120 papers to sift through.

I settle these choices in the plan while they're still easy to change.

2. Ask for a retrieval design

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學



Quick prompt

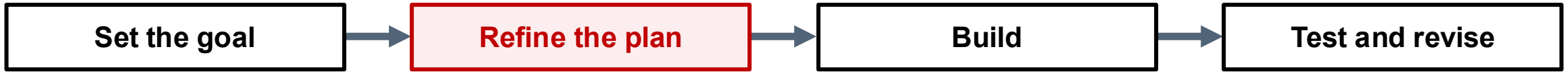
What design decisions do you need me to make before you start coding?

More specific prompt

Recommend a small embedding model that runs on CPU for these 120 papers and tell me what text to embed. Tell me when to compute the paper vectors and where to keep them. Suggest how to score and rank matches, how many to return, and how to handle ties and blank queries. Give me one recommendation for each decision.

3. Specify the retrieval settings

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學



Use **sentence-transformers/all-MiniLM-L6-v2** on CPU.

Batch-encode each title and abstract as one text at startup. **L2-normalize** the paper vectors and keep them in memory.

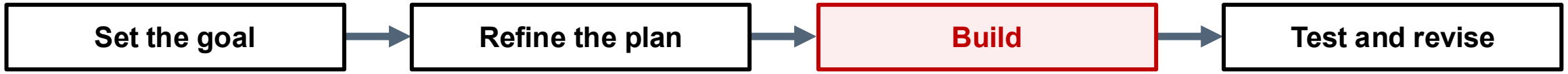
Trim each query and return HTTP 400 if it is blank. Encode it with the same model and normalize its vector. Score every paper with a dot product. Return **up to 10 results** by descending score, then by ascending paper_id for ties.

Update the backend plan with these settings. Wait for my approval.

MiniLM model card

4. Build the backend

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學



1 Build the search API

Implement the retrieval plan using data/papers.json. Load MiniLM once, batch-encode the papers at startup and reuse those vectors for each search. Expose GET /api/search with a q parameter. Return paper_id, title, authors, year, abstract and score in rank order. Start the backend and give me the launch command.

2 Check connectivity

Give me one curl command to test GET /api/search. Use the running backend's host and port and a valid q value. Tell me what HTTP status and response to expect.

5. Frontend from a screenshot

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學

Set the goal → Refine the plan → **Build** → Test and revise

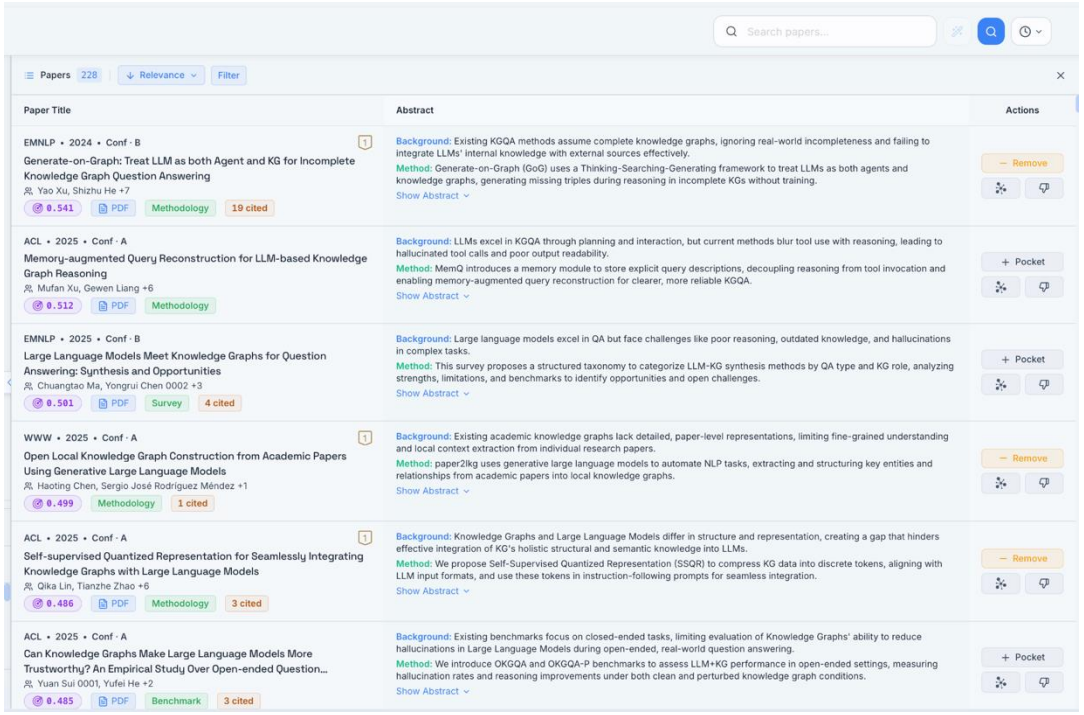
1 Extract UI elements

Extract the layout and list its search controls, result columns and row actions. Match each to the existing API or browser state.

2 Build and connect

Build the interface from that list. Call GET /api/search with q and render the ranked rows and result count. Use browser storage for Save and Remove, and export the saved list as CSV. Start the app and give me the local URL.

Reference screenshot



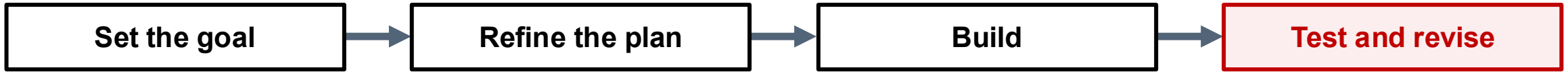
What I want to verify

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學

Goal	Expected behavior
Rank relevant papers	Return up to 10 papers by cosine score with stable ties. Measure Recall@10 against relevance judgments.
Handle blank input	Return HTTP 400 for a blank or whitespace-only query.
Keep a reading list	Saved paper IDs remain after the page reloads.
Export CSV	The CSV contains the saved papers that remain after a removal.

6. Write and run the tests

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學



Write and run tests for corpus loading, retrieval relevance, top-10 ranking and tie handling, API input validation, reading-list persistence and CSV export.

Create your own queries, relevance judgments and edge cases. Set the relevance judgments before running retrieval on data/papers.json.

Store runnable tests in tests/ and fixtures in tests/fixtures/. Put the run commands and results in tests/README.md. Report failures and any checks you couldn’t run.

Homework

THE HONG KONG POLYTECHNIC UNIVERSITY
香港理工大學

A small vector search app

Reuse the demo corpus and record the model and ranking choices.

Deliverables

A runnable app, a reading-list CSV and a short test report.